ARC LABS
ARC CLOUD

Complete Product, System Architecture & Developer Design Plan

Version 1.1 • 14 October 2026 • updated with implementation status

Purpose: A developer-oriented blueprint for a secure IoT cloud platform covering device connectivity, telemetry, dashboards, rules, alerts, APIs and integrations.

Product direction: ARC Cloud is the IoT platform layer. It should operate independently and integrate with ARC LMS/ARC Academy when a learning project needs device telemetry or remote hardware access.

Implementation principle: Deliver a reliable, secure device-to-cloud MVP first; add advanced fleet operations, AI analytics and LMS integration after the core data path is stable.

What changed in v1.1: Sections 1–26 are unchanged from version 1.0. New sections 27–31 record what has been built against each phase of the roadmap (§20), the MVP acceptance criteria (§21), the UI screens (§23) and the repository plan (§22), plus deployment status and the decisions still open.

Status at a glance

Phase	Status	Summary
0 — Discovery	**Owner decision**	Built with documented defaults; owner to confirm (§31).
1 — Foundation	**Done**	Auth, organizations, projects, RBAC, device registry, audit.
2 — Ingestion	**Done**	MQTT over TLS, per-device ACLs, validation, worker, persistence; plus HTTP ingestion.
3 — Console	**Done**	Device list/detail, online/offline, history, 25 dashboard widgets.
4 — Rules	**Done**	Threshold, stale and offline rules; alert lifecycle; email, in-app and webhook notifications.
5 — Integration	**Done**	OpenAPI, webhooks, scoped API keys, service accounts, ARC LMS.
6 — Hardening	**Partly done**	Built and tested; owner sign-off items remain (backups schedule, DPDP review, pen test).
7 — Advanced	**Partly done**	Commands and signed OTA done; billing and native mobile app not built.

1. Executive Summary

ARC Cloud is a multi-tenant IoT platform for connecting embedded devices, collecting telemetry, visualizing live and historical data, managing device identity, and triggering rules or alerts. Initial device families may include ESP32/ESP8266, Raspberry Pi, Arduino-based gateways, industrial sensors and cellular/BLE gateways.

Primary use cases

- Student and prototype projects: sensor readings, device status, MQTT, dashboards and project sharing.
- Industrial monitoring: environment, equipment, energy, smart facilities and asset telemetry.
- Remote gateway/fleet integrations: device health, connectivity and structured telemetry.
- ARC LMS integration: link a learner/course/project to an authorized device or telemetry view without mixing LMS ownership into the IoT data plane.

Product principles

- Secure by default: per-device identity, least privilege, TLS, credential rotation and audit trails.
- Tenant isolation at API, data, device and dashboard levels.
- MQTT-first ingestion with HTTPS APIs for management and integrations.
- Assume messages can be delayed, duplicated or dropped; design retries and idempotent processing.
- Start with a modular backend and split services only when scale or team needs justify it.

2. Goals and Non-Goals

V1 goals

- Register organizations, users, projects, devices and credentials.
- Ingest telemetry through MQTT and optionally HTTPS.
- Store timestamped telemetry and show latest values, history and online/offline state.
- Create dashboards, basic rules, alerts, event logs and scoped API keys.
- Support secure onboarding, credential revocation, audit logs and operational monitoring.

Not required for V1

- Full SCADA/PLC control or safety-critical actuation.
- A general-purpose no-code workflow engine or advanced ML predictive maintenance.
- Native mobile apps before the responsive web console is proven.
- An in-house MQTT broker implementation; use a maintained broker/service.

3. Personas and Roles

Role	Responsibilities
Platform Super Admin	Manage tenants, global settings, service health, plans and audit review.
Organization Admin	Manage members, projects, devices, dashboards, rules and organization settings.
Project Admin / Engineer	Configure devices, telemetry schemas, dashboards, alerts and project integrations.
Operator / Viewer	View permitted devices, dashboards, alerts and events; acknowledge alerts if allowed.
Device Identity	Non-human identity allowed to publish only authorized telemetry and receive permitted commands.
ARC LMS Service Account	Scoped server-to-server access to link learning projects/lab sessions with approved devices or data views.

4. Core Product Modules — V1

Module	V1 scope
Identity & Access	Users, organizations, projects, RBAC, invitations, API keys and service accounts.
Device Registry	Device ID, type/model, firmware, tags, metadata, ownership, status and lifecycle.

Module	V1 scope
Provisioning	Create/claim device, issue unique credentials, rotate/revoke credentials and record events.
MQTT Ingestion	Authenticated connections, topic ACLs, payload validation, timestamp handling and rate limits.
Telemetry Pipeline	Normalize messages, validate schema, enrich tenant/device metadata, persist and record failures.
Time-Series Storage	Measurements indexed by tenant/project/device/metric/time, with retention rules.
Device State	Last seen, online/offline, connection health, firmware and reported state.
Dashboards	Latest value, time-series chart, status, device table, events and thresholds.
Rules & Alerts	Threshold/stale-device rules, alert lifecycle, notification delivery and acknowledgement.
Commands / Downlink	Authorized command request, expiry, acknowledgement and audit history; optional/gated.
Firmware / OTA Metadata	Firmware versions, manifests and release metadata; secure rollout in a later phase.
APIs & Integrations	REST APIs, webhooks, scoped API keys and ARC LMS integration.
Audit & Observability	Device/admin events, metrics, logs, traces, health checks and alerts.

5. Reference Architecture

```
DEVICE LAYER
ESP32 / ESP8266 / Raspberry Pi / Gateway / Industrial Device
↓ MQTT over TLS (primary) • HTTPS (optional)
INGRESS LAYER
Managed MQTT broker or maintained broker cluster + HTTPS ingestion API
↓ authenticated connection • topic ACL • rate limits
PROCESSING LAYER
Ingestion consumer → schema validation → tenant/device lookup → normalization
■■ Device state / last seen
■■ Rule evaluation → alert queue
■■ Telemetry persistence
■■ Event stream / webhook delivery
DATA LAYER
PostgreSQL (metadata) + time-series store + Redis/queue + object storage
↓
APPLICATION LAYER
Web console / dashboards / REST API / webhooks / ARC LMS integration
```

Architecture guidance

- Do not expose the database directly to devices or browsers.
- The broker handles transport; it is not the long-term source of truth.
- Use a durable queue/event stream if message volume or retry requirements demand it.
- Record processing failures and support replay where practical.
- Use server-side authorization for every dashboard query and real-time subscription.

6. Recommended Technology Stack

Layer	Recommendation
Web console	Next.js / React + TypeScript.
Backend API	Node.js + NestJS + TypeScript; REST/OpenAPI.
MQTT	Managed MQTT service or maintained broker such as EMQX/Mosquitto; production HA and ACL strategy required.
Metadata database	PostgreSQL.
Telemetry storage	TimescaleDB/PostgreSQL for MVP if volume fits; evaluate a dedicated time-series store as load grows.
Cache / queue	Redis for cache/jobs; durable queue/stream where delivery guarantees require it.
Object storage	S3-compatible storage for firmware binaries, exports and attachments.
Realtime	WebSocket/SSE gateway backed by authorized server-side updates.
Authentication	OIDC-compatible provider or secure server-side auth; backend-enforced RBAC.
Deployment	Containers, managed database, TLS, secrets management, CI/CD, staging and production.
Observability	Structured logs, metrics, tracing, error tracking and uptime checks.

7. Device Identity, Provisioning & Lifecycle

Every device needs a stable platform device_id and a distinct credential. Never ship a fleet with one shared production username/password or embed organization-wide secrets in firmware.

```
Lifecycle: created → pending_claim → provisioned → active → maintenance → suspended/revoked → retired
```

Provisioning flow

- An authorized admin creates a device or one-time claim code.
- A controlled process provisions a unique credential to the device.
- Device connects using TLS and its own identity.
- Broker ACLs restrict publishing to its own telemetry and permitted command-ack topics.
- Platform records connection/first-message events and updates last_seen.
- Credentials can be rotated, suspended or revoked independently.

```
Required device fields: device_id, organization_id, project_id, display_name, device_type, model,
firmware_version, tags, metadata, lifecycle_status, created_at, last_seen_at and credential_status.
```

8. MQTT Topic & Payload Convention

Proposed topics (finalize before firmware development):

```
Telemetry: arc/v1/{orgId}/{projectId}/{deviceId}/telemetry
State: arc/v1/{orgId}/{projectId}/{deviceId}/state
Command: arc/v1/{orgId}/{projectId}/{deviceId}/cmd
Command acknowledgement: arc/v1/{orgId}/{projectId}/{deviceId}/cmd-ack
```

Example payload

```
{
"schemaVersion": 1,
"messageId": "unique-message-id",
"timestamp": "2026-10-09T10:00:00Z",
"metrics": { "temperatureC": 29.4, "humidityPct": 61.0 },
"metadata": { "firmwareVersion": "1.0.0" }
}
```

- Derive device identity from the authenticated connection/ACL, not from an untrusted payload field.
- Validate payload size, schema, metric types, timestamp range and allowed metric names.
- Store server_received_at separately from device timestamp; use UTC internally.
- Define QoS, retained-message and Last Will behavior for each topic.

9. Online/Offline Logic

Use broker connection events where available plus last-seen timeout as fallback. Make timeout configurable by device class (for example expected reporting interval × 3 with a grace period); do not use one fixed timeout for all devices.

- Online: recent authenticated connection/message within threshold.
- Offline: no connection/message beyond threshold or broker disconnect event.
- Unknown: device has never reported or state cannot be established.
- Show last_seen_at and state source; account for battery-powered and cellular devices.

10. Telemetry Data Model & Retention

Concern	Requirement
Measurement keys	organization_id, project_id, device_id, metric_key, metric_value, device_timestamp, received_timestamp, message_id.
Types	Support numeric measurements first; extend to boolean/string/state when required.
Queries	Latest value, metric history, multi-metric time window, aggregates/downsampling.
Indexes	Tenant/project/device/time; partition by time or use hypertables if volume warrants.
Retention	Configurable raw retention and optional downsampled aggregates per plan.
Quality	Track invalid payloads, missing timestamps, out-of-range values and duplicate IDs.
Isolation	Every query includes authorized tenant/project scope; cross-tenant tests required.

11. Dashboard & Visualization

- Dashboard list/create/edit/duplicate; share only within authorized scope.
- Widgets: latest value, time-series chart, status indicator, device table, event/alert list and threshold gauge where useful.
- Widget configuration: device(s), metric(s), time range, aggregation, units and refresh interval.
- Design empty/loading/error/stale-data states.
- Use server-side authorization for queries and websocket subscriptions.
- Avoid aggressive polling by default; enforce export row/time-range limits.

12. Rules, Alerts & Notifications

V1 rule types: threshold, device stale, online/offline transition. Add rate-of-change after basic rules are reliable.

```
Alert lifecycle: triggered → acknowledged → resolved (timestamps, owner and audit history).
```

- Prevent alert storms with cooldowns, deduplication and recovery conditions.
- Start with in-app and email; add SMS/WhatsApp through approved integrations later.
- Retry failed delivery with bounded backoff and expose repeated failures.
- Run rule evaluation and notification delivery asynchronously where possible.

13. Commands & OTA Safety

Commands can change physical-world behavior. Keep downlink disabled unless the use case and safety model are explicitly approved.

- Authorize commands by tenant, project, device and role.
- Include command_id, requested_by, created_at, expires_at, payload and status.
- Track queued, published, acknowledged, failed and expired states.
- Never report success until the device acknowledgement confirms it.
- Allow-list command types and validate payloads.
- OTA must verify firmware authenticity/integrity, compatibility and rollout status; never trust an unsigned manifest alone.

14. REST API Design

Use /api/v1, consistent pagination, validation, stable error codes and OpenAPI documentation.

Area	Example endpoints
Organizations	GET/POST /api/v1/organizations; GET/PATCH /api/v1/organizations/{id}
Projects	GET/POST /api/v1/projects; GET/PATCH /api/v1/projects/{id}
Devices	GET/POST /api/v1/devices; GET/PATCH /api/v1/devices/{id}; POST /api/v1/devices/{id}/revoke
Provisioning	POST /api/v1/devices/{id}/claim-codes; POST /api/v1/devices/{id}/rotate-credentials
Telemetry	GET /api/v1/devices/{id}/telemetry?metric=temperatureC&from;=...&to;=...
State	GET /api/v1/devices/{id}/state; GET /api/v1/devices?status=offline
Dashboards	GET/POST /api/v1/dashboards; PATCH /api/v1/dashboards/{id}
Rules/alerts	GET/POST /api/v1/rules; GET /api/v1/alerts; POST /api/v1/alerts/{id}/acknowledge
Commands	POST /api/v1/devices/{id}/commands; GET /api/v1/commands/{commandId}
Integrations	POST /api/v1/webhooks; POST /api/v1/api-keys; DELETE /api/v1/api-keys/{id}
Health	GET /health/live; GET /health/ready (internal/protected as appropriate)

15. Core Database Entities

Entity	Important fields / notes
organizations	id, name, slug, status, plan_id, created_at

Entity	Important fields / notes
users	id, identity_provider_id, email, display_name, status
organization_members	organization_id, user_id, role, joined_at
projects	id, organization_id, name, description, timezone, settings
devices	id, organization_id, project_id, name, type, firmware_version, status, last_seen_at, metadata
device_credentials	device_id, credential hash/secret reference, status, created/rotated/revoked timestamps
device_events	device_id, event_type, event_time, details
telemetry_measurements	tenant/project/device IDs, metric_key, value, device_time, received_time, message_id
metric_definitions	project_id, metric_key, label, data_type, unit, validation rules
dashboards/widgets	dashboard scope, widget type, query config, layout
rules/alerts	condition, threshold, cooldown, enabled, status and lifecycle timestamps
notifications	alert_id, channel, delivery status and attempts
commands	device_id, command type/payload/status/requester/expiry
api_keys/webhooks	scopes, key hash, expiry/revocation, subscribed events and secret reference
audit_logs	actor, tenant, action, resource, timestamp and metadata

Data modeling notes

- Use non-guessable IDs for public resources.
- Store credential/API-key hashes or secret-manager references, never plaintext secrets.
- Define tenant data deletion/retention and use migrations for schema changes.
- Use JSON metadata only for variable attributes; keep commonly queried fields typed/indexed.
- Keep telemetry storage separate from operational metadata.

16. ARC LMS / ARC Academy Integration

ARC Cloud remains a separate IoT service. ARC LMS should not query the telemetry database directly.

```
Flow: ARC LMS project/lab session → scoped server-to-server authorization → ARC Cloud API → authorized device/dashboard/telemetry result
```

- Use OAuth2 client credentials or signed service tokens with narrow scopes.
- Map LMS project/lab IDs to ARC Cloud organization/project/device IDs explicitly.
- Use time-limited access for learner-assigned labs where required.
- Never expose device credentials or broker-admin credentials to learners.
- Log cross-platform access and support revocation.

17. Security & Privacy

- TLS for web, API, MQTT and internal links where supported.
- Unique per-device credentials; rotation and revocation.
- No anonymous production MQTT; topic ACLs scoped to device/project.
- Backend RBAC and tenant/project checks on every read/write.
- Rate limits and payload-size limits on API and ingestion.
- Input/schema validation, secure file handling and dependency scanning.
- Secrets manager/environment secrets; no secrets in source control or frontend bundles.
- Audit provisioning, credential changes, rule changes, commands and exports.
- Backups and tested restore; incident response and data export/deletion process.
- Review applicable Indian privacy/legal requirements before production.

18. Reliability & Observability

- Track MQTT connections, publish rate, accepted/rejected messages, queue lag, processing latency and storage errors.
- Track API latency/error rate, database pool, cache health and notification failures.
- Use health/readiness checks, structured logs and actionable alerts.
- Use bounded retries/backoff and dead-letter handling where supported.
- Set per-plan quotas for message rate, payload size, devices, dashboards and retention.
- Set performance targets only after estimating device count and message frequency.
- Document backup RPO/RTO and test recovery.

19. Deployment & Operations

- Maintain local, staging and production environments with separated data/secrets.
- Use versioned deployment configuration and infrastructure as code where practical.
- Run database migrations as controlled release steps.
- Configure TLS, DNS, firewall rules and least-privilege service identities.
- Monitor backup success and test restoration.
- Write runbooks for broker outage, ingestion backlog, database exhaustion, credential compromise and restore.
- Use staged rollout and rollback procedures.

20. Development Roadmap

Phase	Scope	Exit criteria
0 — Discovery	Confirm scale, reporting interval, payload, retention, cloud budget, commands and security model.	Approved PRD, architecture, topic spec, data model and wireframes.
1 — Foundation	Auth, organizations, projects, RBAC, device registry, audit logs.	Secure device/project management; tenant isolation tests pass.
2 — Ingestion	MQTT TLS, per-device identity/ACLs, validation, worker and persistence.	ESP32 publishes reliably; invalid payloads rejected/logged.
3 — Console	Device list/detail, last seen, online/offline, history and dashboard widgets.	Authorized user can inspect current/historical data.
4 — Rules	Threshold/stale rules, alert lifecycle and email/in-app notifications.	Deduplication, recovery and retries pass tests.
5 — Integration	REST APIs, webhooks, API keys and ARC LMS service integration.	Scoped access and audit verified end-to-end.
6 — Hardening	Monitoring, quotas, backup/restore, security review, load tests and runbooks.	Operational readiness checklist signed off.
7 — Advanced	Commands, secure OTA, advanced analytics, billing and mobile UX.	Only built against validated use cases.

21. MVP Acceptance Criteria

- Admin can create a project and register a device.
- Device authenticates to MQTT over TLS with unique credentials.
- Device publishes a documented JSON telemetry payload to an authorized topic.
- Platform validates, timestamps and persists accepted messages.
- Unauthorized topics, revoked credentials and malformed payloads are rejected.
- Console shows state, last seen, latest metrics and historical charts.
- Tenant/project permissions prevent cross-tenant data access.
- Authorized user can create a threshold/stale rule and receive an alert.
- Alerts can be acknowledged/resolved with audit history.
- API is documented, authenticated, scoped and rate-limited.
- Logs/metrics reveal ingestion failures and queue/storage health.
- Backups can be restored through a tested procedure.

22. Suggested Repository Structure

```
apps/
console/ # Next.js web console
api/ # NestJS management/query API
ingestion-worker/ # MQTT validation and persistence
rules-worker/ # rule evaluation and alert processing
packages/
shared-types/ # shared schemas/types
validation/ # payload/API schemas
ui/ # reusable UI components
infra/
docker/ # local development
terraform/ # optional infrastructure as code
docs/
architecture/ # diagrams and decisions
api/ # OpenAPI specification
device-protocol/ # topics, payloads, provisioning
runbooks/ # operational procedures
```

23. Initial UI Screens

Screen	Minimum content
Organization dashboard	Device counts by state, recent alerts, telemetry activity and project shortcuts.
Projects	Project list, environment, devices, members and dashboard links.
Device inventory	Search/filter by status/type/tags; last seen, firmware and lifecycle state.

Screen	Minimum content
Device detail	Identity/status, last seen, charts, events, firmware metadata and permitted actions.
Dashboard builder	Configure widgets, device/metric, time range and refresh interval.
Rules & alerts	Rules list, enabled state, alert history, acknowledgement and resolution.
Integrations	API keys/scopes, webhooks, service accounts and delivery status.
Administration	Members, roles, quotas, audit logs and platform settings.

24. Definition of Done

- Acceptance criteria and error states documented.
- Backend authorization and tenant isolation implemented and tested.
- Input validation and audit requirements met.
- Unit/integration tests cover critical paths.
- API/UI documentation updated.
- Logs and metrics added for important flows.
- Deployment/migration and rollback impact reviewed.
- No secrets or sensitive telemetry exposed in client logs or URLs.

25. Decisions Required at Developer Kickoff

- Expected device count at launch, after 12 months and after 3 years.
- Publish interval, payload size, peak bursts and offline buffering needs.
- Raw telemetry retention and need for aggregates/downsampling.
- Preferred cloud provider/region and monthly infrastructure budget.
- Whether V1 needs downlink commands or OTA, or telemetry-only is sufficient.
- Mandatory alert channels.
- Initial customer: ARC LABS internal, college, industrial customer or multiple tenants.
- Exact data ARC LMS may access from ARC Cloud.
- Required uptime, recovery objectives and operational support owner.

26. Final Recommendation

Build in this order: secure identity and device registry → MQTT ingestion → telemetry persistence → device state and dashboards → alerts → API/LMS integration → production hardening → commands/OTA/advanced analytics.

Do not start with an elaborate dashboard or AI layer before device identity, topic authorization, data model and ingestion are agreed. These decisions underpin security, reliability and future scale.

Document owner: ARC LABS
Audience: Product owner, IoT/firmware engineer, backend/frontend developers, DevOps/cloud engineer and security reviewer
Status: Initial developer blueprint; validate scale, cloud provider, retention and cost during discovery.

27. Implementation Status by Phase

Status as of 14 October 2026, measured against the roadmap in §20. “Done” means built, covered by automated tests, and exercised in a browser against a running system. Evidence points to the repository (github.com/techarclab/Arc_Labs-Cloud).

Phase	Exit criteria (§20)	Status	Evidence
0 — Discovery	Approved PRD, architecture, topic spec, data model and wireframes.	**Owner decision**	Architecture, topic spec and data model documented (docs/architecture, docs/device-protocol, infra/db/migrations). Kickoff answers recorded as defaults in docs/architecture/kickoff-decisions.md, awaiting owner approval.
1 — Foundation	Secure device/project management; tenant isolation tests pass.	**Done**	End-to-end tests: identity, tenancy, provisioning, cross-tenant access returns 404.
2 — Ingestion	ESP32 publishes reliably; invalid payloads rejected/logged.	**Done**	E2E: MQTT/TLS auth, topic ACLs, dedupe, rejection log. Load test 200 devices, 11,800/11,800 messages stored, p95 1.06 s. Real-ESP32 run pending (§30).
3 — Console	Authorized user can inspect current/historical data.	**Done**	Console screens (§29), browser-tested on desktop and phone, viewer role checked.
4 — Rules	Deduplication, recovery and retries pass tests.	**Done**	E2E: one alert per breach, cooldown, recovery, stale and offline rules, email and signed webhook with retries.
5 — Integration	Scoped access and audit verified end-to-end.	**Done**	E2E: channel-scoped read-only key, OAuth2 service account, webhook delivery log, LMS learner token limited to linked devices.
6 — Hardening	Operational readiness checklist signed off.	**Partly done**	Rate limits, quotas, retention, backup + verified restore, CI, load test, runbooks, security review done. Sign-off items in §31.
7 — Advanced	Only built against validated use cases.	**Partly done**	Commands with acknowledgement and signed staged OTA done and tested; usage analytics basic; billing and mobile app not built.

27.1 Phase 0 — Discovery

Plan scope: Confirm scale, reporting interval, payload, retention, cloud budget, commands and security model.

Built

- Topic specification and JSON payloads (docs/device-protocol/README.md); data model as SQL migrations 001–003.
- Architecture decisions (docs/architecture/decisions.md).
- Every §25 kickoff question answered with a working default (docs/architecture/kickoff-decisions.md), e.g. 5–60 s reporting, 90/365/1,825-day retention by plan, RPO 24 h / RTO 1 h.

Verified: Defaults are implemented as settings, so changing an answer is configuration, not a rebuild.

Remaining: Owner approval of the defaults (§31).

27.2 Phase 1 — Foundation

Plan scope: Auth, organizations, projects, RBAC, device registry, audit logs.

Built

- Email/password accounts (scrypt), JWT sessions; organizations, members and roles (Admin, Engineer, Viewer).
- Projects (“channels”) with up to 16 numbered fields, sensor or control.
- Device registry: type, model, firmware, tags, lifecycle (provisioned, active, maintenance, suspended, revoked, retired), unique credentials, rotation and revocation.
- Audit log of every change (provisioning, credentials, rules, commands, keys, settings).

Verified: E2E checks for registration, roles, tenant isolation (resources outside an organization return 404), rotation and revocation.

27.3 Phase 2 — Ingestion

Plan scope: MQTT TLS, per-device identity/ACLs, validation, worker and persistence.

Built

- EMQX 5.8 broker, TLS 1.2/1.3 on 8883; every connection authenticated and every topic authorized by the API.
- Ingestion worker: shared subscriptions, schema and per-field validation, message-ID dedupe, rejected-message log, per-device rate limit, connect/disconnect events.
- PostgreSQL storage (TimescaleDB when available); online/offline from broker events and reporting interval.
- ThingSpeak-style HTTPS ingestion (/api/v1/update?api_key=…&field1=…) for simple boards.

Verified: E2E ingestion, ACL and dedupe checks; load test 200 devices × 1 msg/s for 60 s, nothing lost, p95 latency 1.06 s.

Differs from the plan: HTTP ingestion added alongside MQTT. On Railway the database is plain PostgreSQL (TimescaleDB on the AWS path).

Remaining: One test with a physical ESP32 against the hosted broker.

27.4 Phase 3 — Console

Plan scope: Device list/detail, last seen, online/offline, history and dashboard widgets.

Built

- React web console with organization overview, channels, device list and detail, heartbeat strip, history (1 h to 7 d), CSV export.
- Dashboard builder with 25 widget types (values, gauges, charts, heatmap, map, table, switches, sliders and other controls).
- Team management, rejected-data view, audit log; works on phones.

Verified: Browser tests of every screen on desktop and phone; viewer role sees no edit controls.

Differs from the plan: Built with React + Vite rather than Next.js (a static site that Vercel hosts directly; no server rendering needed).

27.5 Phase 4 — Rules

Plan scope: Threshold/stale rules, alert lifecycle and email/in-app notifications.

Built

- Rules per channel: threshold (with “for N minutes”), no data (stale), and offline; severity, quiet period (cooldown), email recipients.
- Alert lifecycle triggered → acknowledged → resolved, with notes, actor and timestamps; automatic recovery.
- Notifications: in-app (alert list and sidebar count), email via SMTP, signed webhooks; bounded retries with back-off; failures visible per alert.

Verified: E2E: deduplication (one alert per breach), cooldown, recovery, stale and offline rules, email and webhook delivery, retries on failure.

Differs from the plan: Rules run as scheduled jobs inside the ingestion worker instead of a separate rules-worker service; they can be split out by setting JOBS_ENABLED.

Remaining: Set SMTP_URL on the hosted worker to turn on email (in-app and webhooks work without it).

27.6 Phase 5 — Integration

Plan scope: REST APIs, webhooks, API keys and ARC LMS service integration.

Built

- REST API under /api/v1 with OpenAPI 3 documentation at /api/docs (65 paths, also in docs/api/openapi.json).

- API keys (read-only or read/write, limited to chosen channels, expiring) and service accounts (OAuth2 client credentials).
- Webhooks for 10 event types, HMAC-SHA256 signatures, delivery log, test button, private addresses refused.
- ARC LMS: course/lab links to channels; the LMS server issues short-lived read-only learner links; learners use a lab view and never see device credentials.

Verified: E2E: key scope and revocation, OAuth2 token, webhook signing and retries, learner token limited to linked devices.

Remaining: Agree with the ARC LMS team which pages call the learner-token endpoint (docs/integrations/arc-lms.md).

27.7 Phase 6 — Hardening

Plan scope: Monitoring, quotas, backup/restore, security review, load tests and runbooks.

Built

- Rate limits (300 req/min per client, 10/min login), payload limits, per-device message limits.
- Plans and quotas (devices, channels, rules, keys, webhooks) and per-organization data retention; ARC LABS admin page for plans.
- Health and readiness checks, worker metrics, structured logs; backup and restore scripts with checksum and row-count verification.
- CI on every push (build, unit and E2E tests, audit, secret check); load test; runbooks for broker outage, ingestion backlog, database exhaustion, credential compromise, restore and rollback; security review (docs/security/review.md).

Verified: Restore verified in CI; quotas, retention and plan changes covered by E2E.

Remaining: Owner sign-off: turn on Railway scheduled backups, DPDP Act review, penetration test, and a separate staging environment (guide in docs/deploy/staging.md).

27.8 Phase 7 — Advanced

Plan scope: Commands, secure OTA, advanced analytics, billing and mobile UX.

Built

- Commands: per-channel allow-list with typed settings, role check, expiry; states queued → published → acknowledged / failed / expired; success only after device acknowledgement; MQTT and HTTP devices.
- OTA: Ed25519-signed releases (server verifies the signature), SHA-256 checked on the device, staged rollouts by percentage with widen, pause, cancel and per-device progress; ESP32 example firmware.
- Usage analytics: messages per day, busiest devices, alerts by severity.

Verified: E2E: allow-list, queued-while-offline delivery, acknowledgement, expiry, device-reported errors, signed and mis-signed releases, staged rollout completion.

Remaining: Billing and payments, a native mobile app (the console is responsive), and advanced/AI analytics are not built; per §20 they wait for validated use cases.

28. MVP Acceptance Criteria — Status

Criterion (§21)	Status	How it was verified
Admin can create a project and register a device.	**Done**	E2E and browser test.
Device authenticates to MQTT over TLS with unique credentials.	**Done**	E2E against broker auth hooks; real ESP32 on hosted broker pending.
Device publishes a documented JSON telemetry payload to an authorized topic.	**Done**	Protocol in docs/device-protocol; E2E.
Platform validates, timestamps and persists accepted messages.	**Done**	E2E and load test (0 lost of 11,800).
Unauthorized topics, revoked credentials and malformed payloads are rejected.	**Done**	E2E; rejections listed in the console.
Console shows state, last seen, latest metrics and historical charts.	**Done**	Browser test.
Tenant/project permissions prevent cross-tenant data access.	**Done**	E2E (404 for other organizations, scoped keys).
Authorized user can create a threshold/stale rule and receive an alert.	**Done**	E2E and browser test.
Alerts can be acknowledged/resolved with audit history.	**Done**	E2E and browser test.
API is documented, authenticated, scoped and rate-limited.	**Done**	OpenAPI at /api/docs; E2E.
Logs/metrics reveal ingestion failures and queue/storage health.	**Done**	Worker /metrics, rejected-data view, delivery log.
Backups can be restored through a tested procedure.	**Done**	infra/backup verified in CI; schedule on Railway is an owner action.

All 12 criteria are met in the codebase: 60/60 end-to-end checks and 27 unit tests pass, and the console was exercised in a browser (create rule, acknowledge alert, send command, create key and webhook, LMS preview, OTA rollout, retention).

29. Initial UI Screens — Delivered

Screen (§23)	Delivered as	Status
Organization dashboard	Overview: device counts by state, alerts needing attention, messages per hour, channels with online bars.	**Done**
Projects	Channels in the sidebar, each with Dashboard, Devices, Rules, Commands, Channel settings, API keys & export.	**Done**
Device inventory	Devices tab: search, online/offline/not-seen counts, 30-minute heartbeat, latest readings, firmware.	**Done**
Device detail	Readings, history chart, details, commands, activity, credentials, maintenance/suspend/revoke.	**Done**
Dashboard builder	25 widget types, field and options per widget, drag to reorder, per-device view.	**Done**
Rules & alerts	Channel Rules tab; Alerts page with filters, acknowledge/resolve, history and notification status.	**Done**
Integrations	API keys and service accounts, webhooks with delivery log, ARC LMS links.	**Done**
Administration	Team and roles, Usage & limits (quotas, retention), Audit log, ARC LABS admin (all organizations and plans), Firmware & OTA.	**Done**

30. Repository and Deployment — As Built

Plan (§22)	As built
apps/console (Next.js)	apps/console — React + Vite, hosted on Vercel.
apps/api (NestJS)	apps/api — NestJS; REST, OpenAPI, broker auth/ACL hooks on a private port.
apps/ingestion-worker	apps/ingestion-worker — ingestion plus rules, notifications, commands, OTA tracking and retention jobs.
apps/rules-worker	Folded into the worker (can run as a separate instance).
packages/shared-types, validation, ui	packages/shared — topics, schemas, ACL, rule evaluation, connectivity.
infra/docker, terraform	docker-compose.yml, Dockerfiles, infra/emqx, infra/db/migrations, infra/backup; Terraform not used (Railway).
docs/architecture, api, device-protocol, runbooks	All present, plus deploy (Railway/Vercel, staging, AWS), integrations, security, operations.
(not in plan)	firmware/esp32-arduino (MQTT and HTTP examples with OTA), tools (simulator, firmware signing, deploy and diagnostics), tests (E2E, load, UI demo).

Component	Where	Status
Web console	Vercel — arc-labs-cloud.vercel.app	**Done**
API, MQTT broker, worker, PostgreSQL	Railway (US East); devices connect to the TCP proxy on port 8883 (MQTT/TLS) or HTTPS	**Pending deploy**
Code	GitHub — techarclab/Arc_Labs-Cloud (private)	**Done**

Pending deploy: run tools/railway-deploy.sh once to apply the newest settings (console address, ARC LABS admin) and confirm sign-up and one real ESP32 end to end.

31. Open Items and Owner Decisions

Item	Owner	Notes
Confirm kickoff defaults (§25)	ARC LABS	docs/architecture/kickoff-decisions.md — scale, budget, retention, uptime targets.
Redeploy backend and verify live sign-up	ARC LABS	tools/railway-deploy.sh, then tools/railway-doctor.sh if anything fails.
Connect one physical ESP32	ARC LABS	firmware/esp32-arduino/arc_http (HTTPS) or arc_device (MQTT/TLS).
Turn on alert emails	ARC LABS	Set SMTP_URL on the worker (e.g. a Gmail app password or Brevo).
Scheduled database backups	ARC LABS	Enable in Railway; restore procedure in docs/runbooks/backup-restore.md.
DPDP Act 2023 review	ARC LABS / legal	Privacy notice, consent, processing agreements with Railway and Vercel.
Penetration test	ARC LABS	Recommended before the first paying customer.
Staging environment	ARC LABS	Guide in docs/deploy/staging.md.
Vercel account billing	ARC LABS	Team plan showed a suspension notice; Hobby plan is for non-commercial use.
Billing, mobile app, advanced analytics	ARC LABS	Phase 7 items to build once use cases are validated.